

SIGMA Clermont

School of Engineering

Clermont Auvergne INP

Annual Project
Robotic Bin Picking for Industrial components

Students:

Ben Slimene Farah

Jasser Bougatef

Abdelmounim Farouk

Yassine Bour

Julien Navet

Dina Moutih

mai 2026

Contents

1 Introduction **4**

2 Vision System: Shoe and Hole Segmentation using YOLOv11 **5**

2.1 Objective and Model Selection 5

2.2 Initial Dataset Preparation and Configuration 5

2.3 Instance Segmentation Strategy 5

2.3.1 Annotation Procedure 6

2.4 Initial Model Training and Performance Evaluation 7

2.5 Live Testing and The "False Positive" Challenge 9

2.6 Dataset Upgrading for Industrial Robustness 10

3 Hand-eye Calibration of a UR3e with an Intel RealSense D435 Camera **12**

3.1 Definition 12

3.2 Setup 12

3.2.1 Packages and hardware used 12

3.2.2 Experimental procedure 12

3.3 Results 13

3.3.1 Final Transformation 13

3.3.2 Physical Interpretation 13

3.3.3 Using the eye-in-hand calibration 14

4 Grasping strategy **15**

4.1 Principal component analysis on the mask 15

4.1.1 Why PCA 15

4.1.2 How it works 15

4.1.3 Outputs of this stage 16

4.2 Rim point extraction 16

4.3 Depth lookup with robust fallback 17

4.4 Back-projection into the camera frame 17

4.5 Transformation to the robot base frame 18

4.6 Output . . . 18
4.6.1 Published topics . . . 18

5 Workspace Overview **19**

6 Pick and Place with MoveIt 2 **21**

6.1 Objective of this part . . . 21
6.2 Workspace analysis and anticipated problems . . . 21
6.2.1 Modelling the workshop table as an obstacle . . . 22
6.2.2 Attaching the camera mount . . . 23
6.2.3 Constraining the workspace with a virtual box . . . 24
6.2.4 Pre-defined poses versus the dynamic pick pose . . . 25
6.3 Detailed walk-through of `pick_and_place_grasp.py` . . . 26
6.3.1 Initialisation of MoveIt 2 . . . 26
6.3.2 Step 0 Listening to the grasp detector . . . 26
6.3.3 Steps 1–4 The pick . . . 28
6.3.4 Steps 5–9 The place and the return home . . . 29
6.3.5 Summary of the full state machine . . . 29
6.4 Gripper control: attempted but not finished . . . 30
6.5 Conclusion . . . 31

7 Plane Removal with RANSAC **32**

7.1 Objective . . . 32
7.2 RANSAC Principle . . . 32
7.3 Integration in the Pipeline . . . 33
7.4 Implementation Approach . . . 33
7.5 Limitations . . . 34

8 Appendix A: Commands used for the eye-in-hand calibration **35**

8.1 Terminal 1 — UR3e Driver . . . 35
8.2 Terminal 2 — RealSense Camera . . . 35
8.3 Terminal 3 — ArUco Detection . . . 35
8.4 Terminal 4 — easy_handeye2 Server . . . 35

8.5 Terminal 5 — rqt_calibrator GUI 35

8.6 Terminal 6 — Detection Visualization 36

Appendix B – Commands used for detection and pick-and-place 36

1 Introduction

Robotic bin-picking remains a complex challenge in industrial automation, particularly when handling irregular, deformable objects. The overarching objective of this project is to develop a robust, autonomous robotic system capable of identifying, safely grasping, and relocating industrial components specifically focusing on shoes. By bridging deep learning-based computer vision with advanced robotic kinematics, the system utilizes a UR3e manipulator equipped with an Intel RealSense D435 RGB-D camera in an eye-in-hand configuration. The project's development spans two semesters, evolving from foundational simulation setups to a highly precise, end-to-end manipulation pipeline.

During the first semester, the primary focus was on establishing a stable simulation environment using ROS 2 Humble and Ignition Gazebo. This phase involved integrating the UR3e robotic arm alongside a standalone RGB-D camera. Significant technical hurdles were overcome to ensure reliable data transmission, requiring the implementation of correct system plugins and ROS-Ignition bridges to allow the robot to "see" its environment. Initial perception efforts relied on standard YOLO bounding-box detection. However, live testing revealed that this basic approach was insufficient for millimeter-level robotic gripping and posed critical safety risks, as the model occasionally generated "false positives" by confusing human hands or robotic hardware with the target shoe.

To achieve industrial-grade reliability, the second semester completely overhauled the perception and grasping strategy. The vision system was upgraded to a YOLOv11s-seg Instance Segmentation model, trained on a heavily expanded dataset containing negative backgrounds to mathematically teach the AI what not to detect. Rather than drawing imprecise boxes, this model generates exact pixel-level polygon masks of the shoe and the target gripping hole. The system then applies Principal Component Analysis (PCA) to these masks to extract the hole's minor axis, calculating the most stable and natural approach angle for a two-finger grasp.

The final phase of the project bridged this advanced visual intelligence with physical robotic execution. Through a rigorous hand-eye calibration process, the 2D pixel targets and depth data are transformed into 3D coordinates mapped perfectly to the robot's base frame. Using MoveIt 2, these coordinates are translated into a safe, ten-step pick-and-place trajectory. The motion planner dynamically restricts the robot's operating volume and factors in attached collision objects like the camera mount and the workspace table to prevent hardware damage. Tested and validated using custom 3D shoe models within the Gazebo environment, the resulting pipeline successfully translates raw visual data into fluid, autonomous robotic manipulation.

2 Vision System: Shoe and Hole Segmentation using YOLOv11

2.1 Objective and Model Selection

The primary objective of the vision system is to provide the UR3e robotic arm with accurate spatial coordinates to successfully grip a shoe. For this task, simple bounding boxes (Object Detection) were insufficient because the robotic gripper could collide with the irregular edges of the shoe. Therefore, we opted for Instance Segmentation. We selected the state-of-the-art **YOLOv11s-seg** (Small Segmentation) model. This specific architecture provides an optimal balance between high inference speed (crucial for real-time robotic control) and high mask precision.

2.2 Initial Dataset Preparation and Configuration

The foundation of any robust AI model lies in the quality, accuracy, and relevance of its training data. We initiated the project by constructing a custom dataset from scratch. We captured numerous images of various shoes directly within the UR3e robotic arm's actual workspace. This environmental consistency ensured the model would learn under the specific lighting conditions, camera angles, and backgrounds it would encounter during real-world operation.

To process and manage this raw data, we utilized the Roboflow platform. During the manual annotation phase, we defined two strict classes:

- **`shoe:`** Representing the main physical body of the object, which acts as an obstacle that the robot must not hit.
- **`hole:`** Representing the precise target coordinate for the robotic gripping or insertion action.

2.3 Instance Segmentation Strategy

To ensure safe and precise robotic manipulation, a fine-grained understanding of object geometry is required. The UR3e robotic arm operates with millimeter-level precision, making standard object detection approaches based on rectangular bounding boxes insufficient. Bounding boxes inherently include irrelevant background regions, which can lead to inaccurate spatial interpretation and potential collision risks.

To overcome these limitations, an instance segmentation approach was adopted. Instead of coarse bounding boxes, precise polygonal masks were manually annotated for each object. These masks tightly follow the exact contours of both the shoes and the holes, capturing detailed shape information at the pixel level.

The annotated dataset was then processed and managed using Roboflow, which facilitated data organization, visualization, and export for model training., where each object instance was carefully outlined to ensure high-quality labels. Although time-consuming, this step is critical

for achieving accurate perception in robotic applications.

The resulting dataset enables the training of a YOLOv11s-seg model capable of predicting precise segmentation masks. This allows the system to extract accurate spatial information, improving both collision avoidance and target localization during robotic operation.

2.3.1 Annotation Procedure

To generate high-quality segmentation labels, the annotation process was carried out using the *makesense.ai* platform. A total of approximately 1200 images were manually annotated.

Once annotated, the dataset was imported into Roboflow for further processing and export.

Each image was labeled with the two classes: shoe and hole.

Rather than using bounding boxes, polygonal segmentation masks were selected as the annotation format. This choice allows for precise contouring of both the outer shape of the shoe and the inner geometry of the hole, ensuring accurate spatial representation.

Each object instance was carefully outlined point by point to tightly match its boundaries. This level of precision is essential for enabling reliable robotic interaction, particularly for collision avoidance and accurate target localization.

Once the entire dataset was annotated, all labels were exported in a single **COCO JSON** file format. This standardized format facilitates seamless integration with deep learning frameworks and was directly used for training the YOLOv11s-seg model.

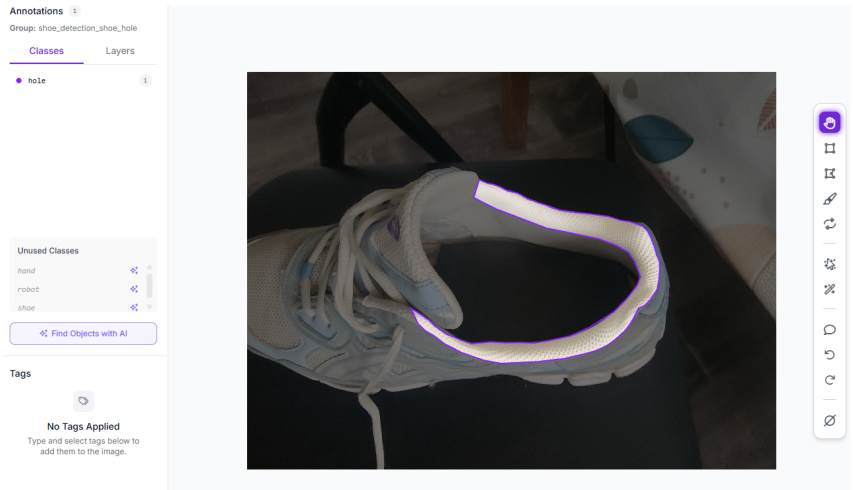


Figure 1: Example of manual instance segmentation in Roboflow. Precise polygon masks outlinethe gripping hole, providing critical geometric data for the robotic arm.

Once the images were fully annotated, the dataset was exported and split into training, validation, and testing sets.



Figure 2: Example of an annotated image in Roboflow, showing the precise polygon masks outlining both the shoe and the gripping hole.

2.4 Initial Model Training and Performance Evaluation

To establish a baseline, we executed our initial training iteration on the restricted dataset. The objective was to verify if the YOLOv11 segmentation architecture could effectively learn the geometric features of our specific target objects.

The implementation was conducted using the `ultralytics` library within the Kaggle cloud environment. We initialized a pre-trained model and fine-tuned it over 300 epochs. We deliberately increased the classification penalty (`cls=10.0`) to force the model to strictly differentiate between the classes.

```
from ultralytics import YOLO
import os
from IPython.display import FileLink

# 1. Initialize the YOLOv11 segmentation model
model = YOLO("yolo11n-seg.pt")

```

```python
# 2. Execute the training phase
results = model.train(
    data="dataset/data.yaml",
    epochs=300,
    imgsz=640,
    cls=10.0  # High penalty to reduce class confusion
)

# 3. Export the trained weights for local deployment
weights_path = "/kaggle/working/runs/segment/train/weights/best.pt"
export_name = "mon_modele_ur3e.pt"

if os.path.exists(weights_path):
    os.rename(weights_path, export_name)
    display(FileLink(export_name))
```

Listing 1: YOLOv11 Training and Export Script used in Kaggle

The generated training metrics (Figure 3) initially suggested an overwhelmingly successful model. We conducted a detailed analysis of the learning curves:

- **Loss Convergence:** The `Box Loss`, `Class Loss`, and `Object Loss` exhibited expected high volatility during the early epochs. However, after approximately epoch 100, all three metrics rapidly converged toward near-zero values, demonstrating that the network's weight adjustments were highly effective in minimizing prediction errors.
- **Mean Average Precision (mAP):** The performance graphs mirror the loss curves. Following the initial instability, the mAP metric stabilized at nearly 100%, and the stricter mAP@50:95 metric plateaued impressively around 90%.



Figure 3: Performance metrics from the initial training. The rapid convergence of loss metrics and the near-perfect mAP after 100 epochs created a deceptive illusion of a flawless model.

Mathematically, the model appeared flawless. It had perfectly mapped the training data. However, relying solely on these theoretical graphs proved dangerously misleading. This "perfect" convergence indicated that the model had completely memorized its limited, binary environment (containing only shoes and holes), leaving it completely unprepared for unstructured, real-world data.

This discrepancy between theoretical perfection and practical application directly triggered the most critical operational challenge of the project during the live testing phase.

2.5 Live Testing and The "False Positive" Challenge

To evaluate the model's performance in real-world conditions, we developed an OpenCV-based Python script capable of capturing the webcam feed and applying the YOLOv11s-seg predictions in real-time.

Critical Safety Issue Identified:
While the system successfully detected the shoe and the hole, a major safety concern emerged: the model falsely detected a hand as a "shoe". Because the initial AI had only been trained on a binary worldview (shoes and holes), it attempted to force any unidentifiable complex shape into its known categories. In an industrial context, this "false positive" could cause the UR3e arm to attempt to grip the operator.

2.6 Dataset Upgrading for Industrial Robustness

Fixing this issue required a complete overhaul of our dataset strategy. Simply increasing the confidence threshold was not a permanent solution. To achieve industrial-level reliability, we significantly expanded and upgraded the dataset to 1200 images, implementing three key strategies:

- **Negative Backgrounds:** We introduced images of empty laboratory environment, the work table, and human hands without any shoes. We paired these with empty `.txt` annotation files. This mathematically taught the AI what *not* to detect.
- **Explicit Class Definitions:** We segmented and added new classes specifically for the operator (`hand`) and the robotic hardware (`robot-arm`) so the AI could explicitly differentiate them from the target object.
- **Perspective Augmentation:** We included photos of shoes captured from various extreme angles to improve the model's 3D generalization capabilities, knowing the robotic arm's camera will view the scene dynamically.

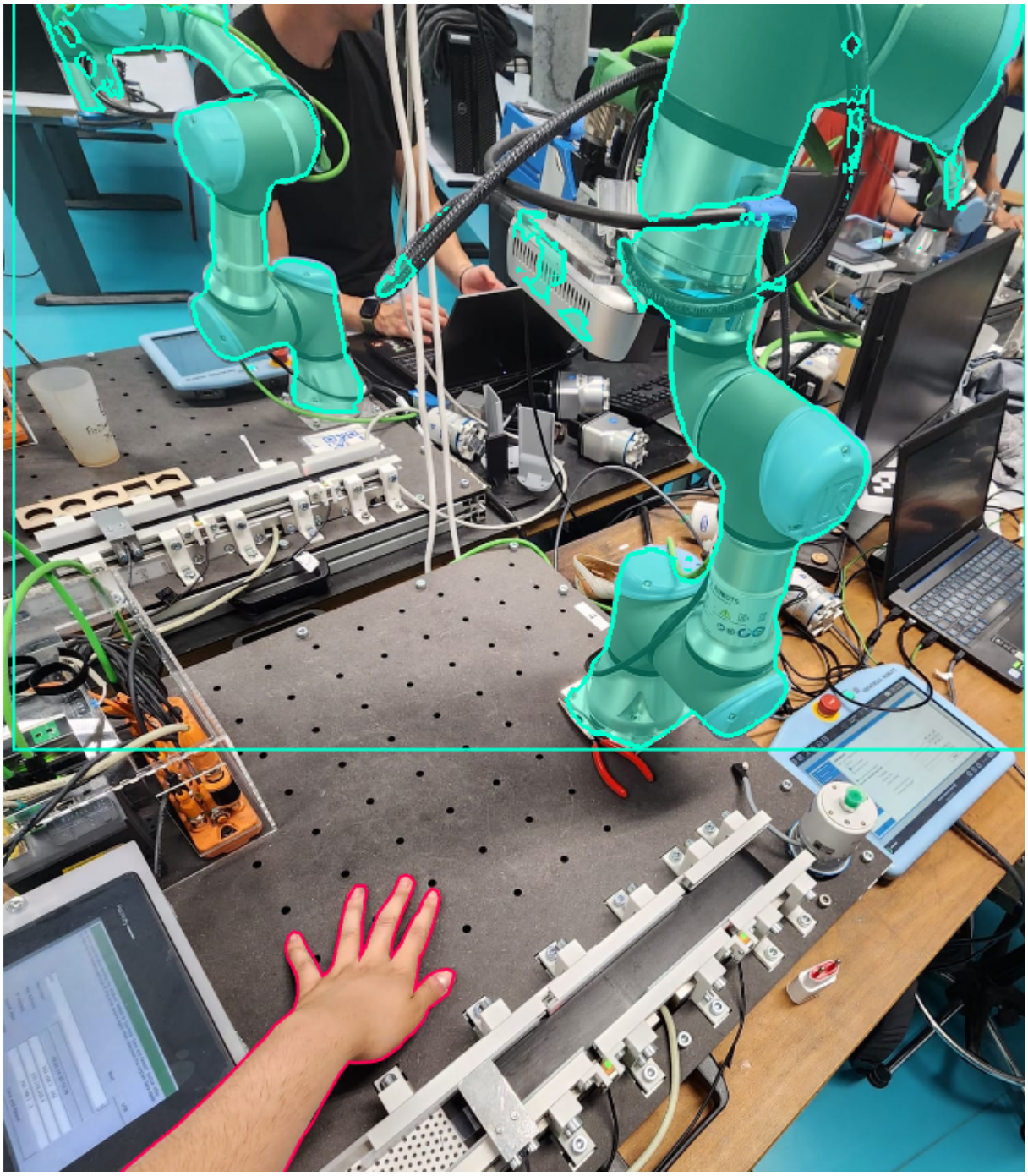


Figure 4: Envrinoment segmentation

The implementation of these dataset engineering strategies directly resolved most of critical false positive issue. By explicitly teaching the YOLOv11s-seg model to recognize and ignore the operator's hands and the robotic arm, most of false detections were virtually eliminated during live testing.

3 Hand-eye Calibration of a UR3e with an Intel RealSense D435 Camera

3.1 Definition

Hand-eye calibration is an important procedure that allows us to transform the camera's coordinate system into the robot's coordinate system. This is because a camera perceives an object in its own coordinate system. We must therefore be able to transform this coordinate system into the robot's coordinate system so that the robot can interact with the object.
In this project, we will use the eye-in-hand configuration. In this configuration, the camera is mounted on the robot's end-effector and moves with it. This configuration notably allows for greater flexibility in relation to the environment being observed.

3.2 Setup

3.2.1 Packages and hardware used

For this task, the ur_robot_driver package is used. This package enables data exchange between the robot and the computer. The realsense2_camera package publishes the information collected by the camera. The aruco_ros package detects the ArUco marker and publishes its pose. Finally, the easy_handeye2 package collects all this information and solves the equation to transform from the camera coordinate system to the robot coordinate system.

3.2.2 Experimental procedure

Calibration was performed by launching six separate terminals. The corresponding commands are provided in the appendix. After attaching the ArUco, position the robot in freedrive mode so that the camera can see the ArUco. A cube and a marker on the ArUco should then appear on the screen, as shown on the right side of this photo:

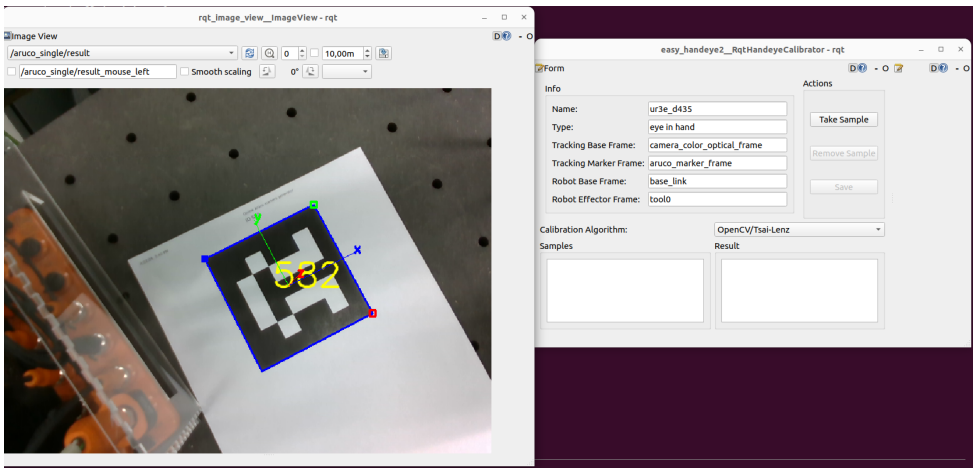


Figure 5: Eye-in-hand calibration

Next, press “Take sample.” This operation must be repeated several times while changing the camera’s orientation and the robot’s configuration (approximately 20 times). Finally, press “Save” to save the calibration results in the file /.ros2/easy_handeye2/ur3e_d435.calib.
This code allows us to transform any point in the camera coordinate system (camera_color_optical_frame) into the robot coordinate system (tool0). After analyzing the measured parameters (translation: x, y, z and rotation: x, y, z, w), the resulting calibration accurately reflects the physical position of the D435 camera on the UR3e end-effector.

3.3 Results

3.3.1 Final Transformation

Once the calibration was complete, we obtained the following transformation from `tool0` to `camera_color_optical_frame`:

```
parameters:
   name: ur3e_d435
   calibration_type: eye_in_hand
   robot_base_frame: base_link
   robot_effector_frame: tool0
   tracking_base_frame: camera_color_optical_frame
   tracking_marker_frame: aruco_marker_frame

transform:
   translation:
     x:  0.0704 m
     y: -0.0379 m
     z:  0.0309 m
   rotation (quaternion):
     x: -0.0062
     y: -0.0006
     z:  0.6941
     w:  0.7198
```

3.3.2 Physical Interpretation

Camera position relative to the `tool0` effector:

- $\Delta x = +7.0$ cm: Forward offset along the `tool0` X-axis.
- $\Delta y = -3.8$ cm: Lateral offset along the negative Y-axis.
- $\Delta z = +3.1$ cm: Vertical offset along the positive Z-axis.

Orientation: The quaternion is dominated by $z = 0.694$ and $w = 0.720$, which points to a rotation around the Z-axis. We can calculate the angle as follows:

$$\theta = 2\arccos(w) = 2\arccos(0.720) \approx 88^\circ \tag{1}$$

These results make sense given our hardware setup. The camera is mounted on a side bracket next to `tool0`, and the image X and Y axes are rotated by roughly 90° compared to those of the effector.

3.3.3 Using the eye-in-hand calibration

Calibration is used in the grasp_detector.py code to determine the position of the grasping point in the robot's coordinate system.

4 Grasping strategy

4.1 Principal component analysis on the mask

4.1.1 Why PCA

Given a binary mask of an arbitrarily oriented hole, the natural parameters of a two-finger grasp are the axes of an inertia ellipse fitted to the mask pixels. The longer (**major**) axis aligns with the elongation of the hole, and the shorter (**minor**) axis is perpendicular to it. Closing the fingers along the minor axis is preferable for several reasons:

- the fingertips travel the shortest distance to reach the rim;
- the material between them is at its widest, giving the most stable grip;
- the resulting grasp is naturally aligned with the principal inertia of the hole, minimising in-grip rotation.

Figure 6 illustrates the geometry on an idealised elliptic hole.

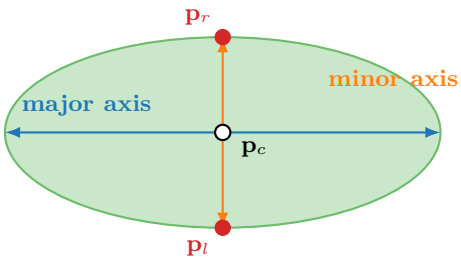


Figure 6: Mask of a hole (green) with its two principal axes computed by PCA.

4.1.2 How it works

PCA is run on the binary mask pixels of the detected hole. It returns two perpendicular axes that describe the shape of the mask:

- the **major axis** the direction in which the mask is most elongated;
- the **minor axis** the direction perpendicular to the major axis, along which the mask is narrowest.

The mask centroid (the average position of all mask pixels) provides the starting point for the rim search in the next stage. Together, the centroid and the two axes fully describe the orientation and extent of the hole in the image.

4.1.3 Outputs of this stage

For every successful detection, Stage 2 produces:

- the centroid pixel of the hole mask;
- the unit vector of the major axis;
- the unit vector of the minor axis;
- the half-extent of the mask along each axis (used in Stage 3 to bound the rim search).

4.2 Rim point extraction

The two fingertip pixels are obtained by walking along the minor axis from the centroid in both directions until the mask boundary is crossed. The first pixel that lies outside the mask, on each side, is taken as a rim point.

Starting at the centroid, the algorithm steps one pixel at a time along the minor axis direction, checking the mask value at each position. The moment the value transitions from inside the mask (255) to outside (0), the algorithm has found the boundary on that side.

Algorithm 1 Rim point extraction along the minor axis
Require: Mask M, centroid $\mathbf{c}$, minor axis $\mathbf{a}$, half-extent h

Initialise left and right rim points to none
for $t = 0, 1, 2, \ldots, h + 5$ **do**
    $\mathbf{q} \leftarrow \mathbf{c} + t\,\mathbf{a}$
    if $M[\mathbf{q}] = 0$ **then**
        right rim point $\leftarrow \mathbf{q}$; **break**
    end if
end for
for $t = 0, 1, 2, \ldots, h + 5$ **do**
    $\mathbf{q} \leftarrow \mathbf{c} - t\,\mathbf{a}$
    if $M[\mathbf{q}] = 0$ **then**
        left rim point $\leftarrow \mathbf{q}$; **break**
    end if
end for
return (left rim point, right rim point)

The step size of one pixel guarantees that the boundary is detected as accurately as the image grid allows. The upper bound $h + 5$ provides a small safety margin in case the mask boundary is slightly noisy.

If either side of the search fails to cross the boundary for example because the mask touches the edge of the image the frame is treated as degenerate and no point is published.

4.3 Depth lookup with robust fallback

Each pixel target centroid, left rim, right rim must be augmented with a depth value to permit back-projection into 3D in Stage 5. The RealSense D435 publishes a depth image aligned to the colour image at `/camera/camera/aligned_depth_to_color/image_raw`. A single pixel of depth is too noisy to be trusted, the implementation therefore samples a small square window around each pixel of interest and takes the median of all non-zero values inside that window, returning the result in metres or marking it invalid if no valid pixel is found.

The implementation therefore summarised in Algorithm 2.

Algorithm 2 Robust depth lookup at the grasp centre

$d_c \leftarrow$ MEDIAN_DEPTH(centroid, small window)
if d_c is valid **then**
    return d_c
end if
if at least one rim point has valid depth **then**
    return average of valid rim depths ▷ rim fallback
end if
$d_c \leftarrow$ MEDIAN_DEPTH(centroid, large window)
if d_c is valid **then**
    return d_c ▷ wide-window fallback
end if
return skip frame

4.4 Back-projection into the camera frame

Each pixel target augmented with a valid depth value is converted to a 3D point in the camera optical frame using the standard pinhole camera model. Conceptually, the model says: given a pixel position and a depth, the corresponding 3D point lies on the ray that passes through the camera centre and that pixel, at the distance given by the depth value.

The model the uses **intrinsic parameters**. These are read once from the `color/camera_info` topic and cached:

```
def camera_info_cb(self, msg: CameraInfo):
    if self.fx is None:
        self.fx, self.fy = msg.k[0], msg.k[4]
        self.cx, self.cy = msg.k[2], msg.k[5]
```

Listing 2: Intrinsics caching

The optical frame convention used by the RealSense driver follows the standard ROS optical-frame definition: X points right, Y points down, Z points along the optical axis (away from the lens). After this stage, the centroid and the two rim points are each expressed as a 3D coordinate in this frame.

4.5 Transformation to the robot base frame

The three points produced by Stage 5 live in the camera optical frame and must be expressed in `base_link` to be useful for motion planning. The transformation chain involves two rigid transforms, depicted in Figure 7: from the camera frame to the robot flange (`tool0`) using the hand-eye calibration , and from `tool0` to `base_link` using the robot's forward kinematics.

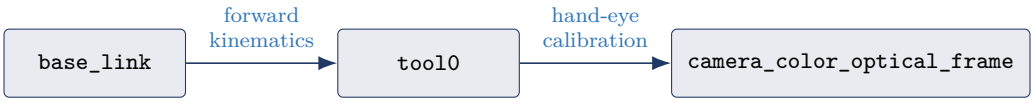


Figure 7: TF chain.

4.6 Output

4.6.1 Published topics

The three resulting points and one debug image are published as:

Topic	Type	Meaning
`/shoe_detector/grasp_point_base`	`geometry_msgs/PointStamped`	TCP target (centre)
`/shoe_detector/grasp_left_base`	`geometry_msgs/PointStamped`	Left fingertip target
`/shoe_detector/grasp_right_base`	`geometry_msgs/PointStamped`	Right fingertip target
`/shoe_detector/image_annotated`	`sensor_msgs/Image`	Visual debug overlay

Table 1: Published topics.

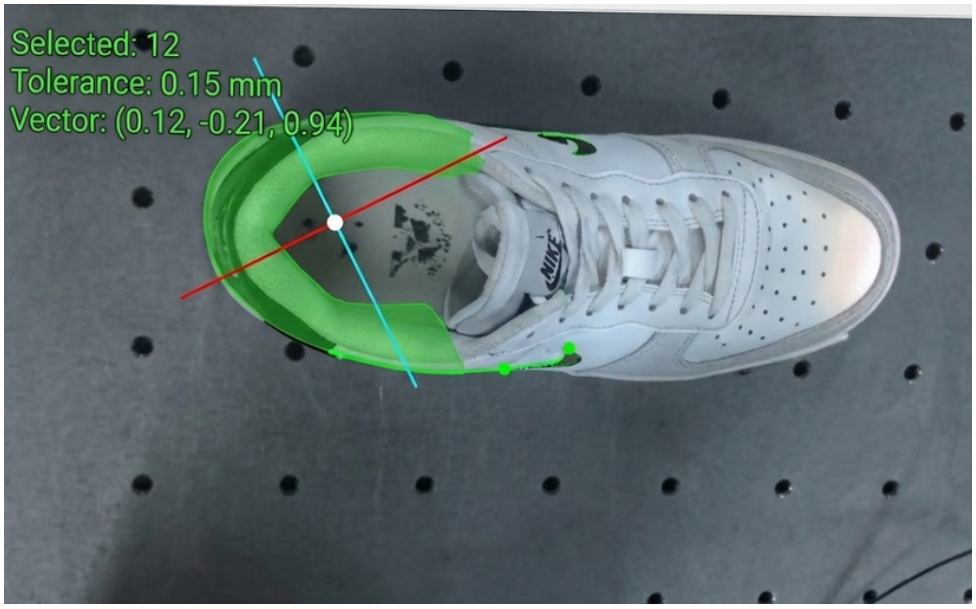


Figure 8: shoe grasping point.

5 Workspace Overview

Before going into the implementation details, it is useful to look at the workspace from a system point of view. Our project is built on top of ROS 2 Humble and is composed of three independent layers that communicate exclusively through ROS topics, services and the TF2 transform tree:

- **Hardware layer.**
  - Universal Robots UR3e arm, driven by the official ROS 2 driver `ur_robot_driver`.
  - Intel RealSense D435 depth camera, driven by `realsense2_camera`.
- **Perception layer** the `shoe_detector` package. It is responsible for:
  - running a YOLO segmentation network on the colour image,
  - locating the lace hole on the shoe and computing a two-finger grasp point,
  - back-projecting that point into 3D using the depth image,
  - publishing it in the robot base frame.
- **Motion layer** the part discussed in this section. It is built around:
  - MoveIt 2 and the `move_group` action server,
  - the Python script `pick_and_place_grasp.py`, which orchestrates the full pick-and-place sequence using the `pymoveit2` client library.

The data flow between these three layers is shown in Figure 9. Reading it from top to bottom:

- the camera publishes colour and depth images,
- the perception node consumes them, computes the grasp point and uses TF2 to transform it from the camera frame to `base_link`,
- the motion script subscribes to that topic, builds the pick poses from it, and sends plan-and-execute requests to `move_group`,
- `move_group` streams the planned trajectories to the UR3e driver,
- the dashed arrow on the right is the `/joint_states` feedback that MoveIt 2 needs to seed every new plan from the actual current pose of the arm.

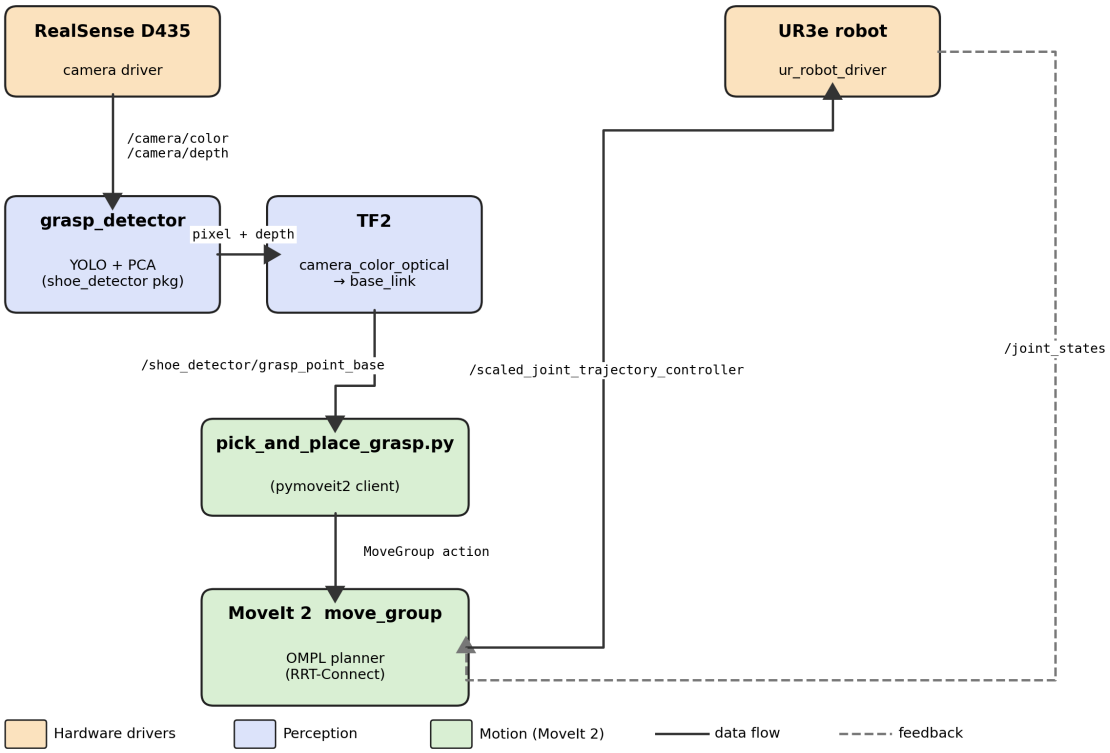


Figure 9: Workspace architecture of the pick-and-place pipeline.

In Figure 9, the orange blocks represent the hardware drivers (the RealSense D435 camera and the UR3e robot driver), the blue blocks form the perception pipeline (`grasp_detector` and the TF2 transform), and the green blocks are the motion stack we are responsible for (`pick_and_place_grasp.py` and the MoveIt 2 `move_group` server). Solid arrows carry forward data images going into the detector, the grasp point going into the motion script, and the planned trajectories going into the robot driver while the dashed arrow on the far right is the `/joint_states` feedback that MoveIt 2 needs to seed every new plan from the actual current pose of the arm.

The strong point of this architecture is that the three layers are fully decoupled:

- the motion script does not import anything from the detector,
- it does not know which neural network is running, nor the camera intrinsics,
- it does not perform any TF lookup itself,
- it only sees a stream of `geometry_msgs/PointStamped` messages on a single topic, already expressed in the robot base frame.

This made debugging much easier in practice, because each member of the team could test their own block in isolation: the perception node could be replayed from a rosbag, and the motion script could be tested with a small fake publisher mocking the grasp point topic.

6 Pick and Place with MoveIt 2

6.1 Objective of this part

The objective of this part of the project is to bridge the gap between the perception side and the physical robot. More precisely, our role in MoveIt 2 is to take the grasp point produced by the detection model (see the previous section on the YOLO-based grasp generator) and transform it into a complete and safe pick-and-place motion: the UR3e must move from a known home configuration to the grasp point published by the detector, perform the pick, then transport the object to a fixed drop location, release it, and finally return home all without colliding with the workshop table, the boxes around it, or itself, and at a velocity that is safe enough to allow manual intervention. In other words, the goal is to make the robot *actually use* the grasp point computed by the vision pipeline, rather than only display it on screen.

This sounds straightforward in principle, but the path from a single 3D point to a reliable motion goes through several non-trivial design choices: how to model the environment so that the planner does not hit something, how to filter the noisy output of the detector, which planning strategy to use for each leg of the trajectory, and how to recover from the inevitable execution failures. The rest of this section walks through the choices we made and the reasoning behind them.

6.2 Workspace analysis and anticipated problems

Before writing a single line of motion code we spent some time in front of the actual robot, looking at the workspace and listing the problems we expected to face. The UR3e is mounted on the corner of a laboratory table that also carries two box-shaped enclosures (the control box of the robot and an auxiliary box used for cabling), and the camera is rigidly mounted on the wrist through a custom 3D-printed support. From this physical setup we identified four sources of potential failure that the motion stack would have to deal with.

1. Static obstacles around the robot. The table itself, and the two boxes on top of it, are well within the reachable workspace of the UR3e. If MoveIt 2 is not told they exist, nothing prevents the planner from generating a trajectory that goes straight through them. The first thing we did was therefore to *model* those obstacles.

2. The camera mount on the wrist. The 3D-printed support that holds the RealSense D435 sticks out from the `tool0` link by a few centimetres. Because it is not part of the official UR3e URDF, MoveIt 2 ignores it by default. During our first dry runs the wrist clipped the camera against the body of the arm in some configurations. We therefore had to measure the mount and add it to the planning scene as an *attached collision object* on the end-effector.

3. Long, "creative" paths from the sampling-based planner. MoveIt 2 uses OMPL as its planning back-end, and the default planner configured for the UR manipulator is **RRT-Connect**,

a sampling-based bidirectional tree planner. RRT-Connect is fast and finds a solution most of the time, but because it explores the configuration space randomly, the resulting paths can be very non-intuitive: the elbow may swing far behind the robot before coming back to the goal. On a robot mounted in a corner this kind of detour is dangerous because it can hit the wall or the boxes. To force the planner to stay in a sensible region we added a virtual bounding box that constrains where the robot is allowed to move.

4. Repeatable poses for the place side. The pick pose is dynamic: it depends on where the shoe is, which is exactly why we are using vision in the first place. The place side on the other hand is fixed we always drop the shoe in the same container and the same is true for the home pose. We took advantage of that and pre-recorded those configurations in joint space, which is both faster to plan and impossible to "reinterpret" by the inverse kinematics solver.

The next subsections describe how we addressed each of these four points.

6.2.1 Modelling the workshop table as an obstacle

The physical workspace was first reconstructed as a set of STL meshes inside the package `src/models/class_table`. The model is composed of one main mesh for the table and three additional meshes for the boxes that sit on top of it (`table.STL`, `BOX1.STL`, `BOX2.STL`, `BOX3.STL`). These meshes were originally built for the Gazebo simulation, where they are referenced from the SDF file `model.sdf` of the `class_table` model, but they also gave us a one-to-one geometric reference of what the planner should treat as non-traversable. Figure 10 shows the corresponding model.

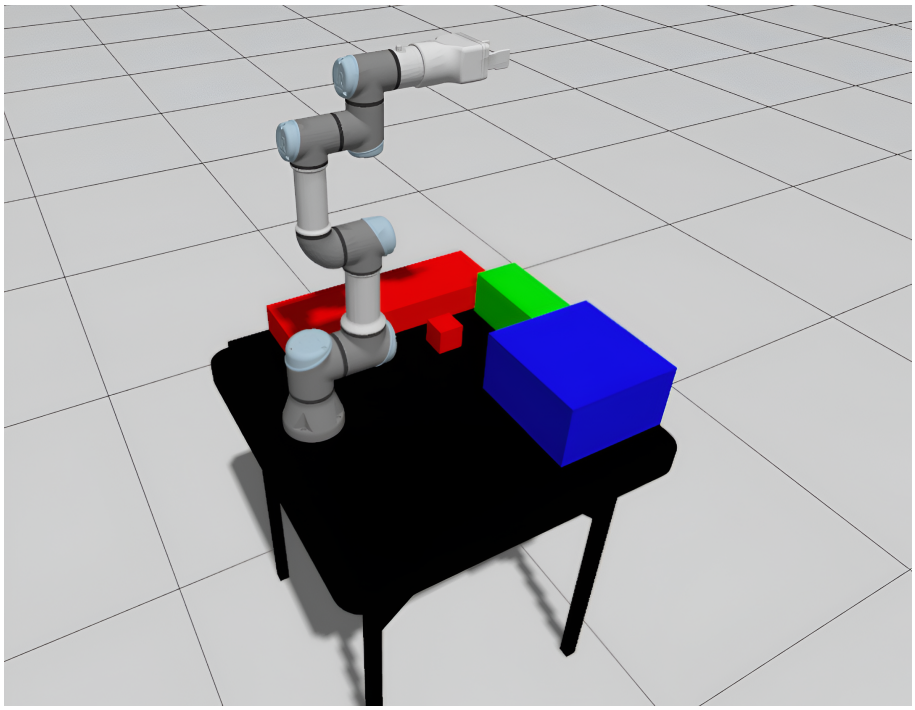


Figure 10: The `class_table` model used to represent the workspace table and the static boxes around the UR3e.

Once the geometry was clear, we introduced it into MoveIt 2 through the *planning scene*. MoveIt 2 maintains an internal world representation in which any object primitive or mesh can be declared as a collision object. From the moment such an object is present in the scene, the OMPL planner takes it into account and any plan whose trajectory passes through the object is rejected. In our case, for stability and because most of the obstacles are axis-aligned, we approximated the workshop table by a thick box placed just below the actual table top at $Z = -0.01\,\mathrm{m}$, with a thickness of 50 cm. The advantage of using a box rather than the full STL mesh in the planning scene is that mesh-based collision checks are significantly more expensive than primitive checks, which slows down the planner; the disadvantage is that we lose the precise shape of the boxes on top of the table, but for our pick-and-place trajectories the simplified shape was conservative enough.

The corresponding code is concentrated in the helper `setup_collision_scene`:

```
def setup_collision_scene(arm: MoveIt2) -> None:
    for name in ["table", "ceiling", "floor"]:
        arm.remove_collision_object(name)
    time.sleep(0.4)

    arm.add_collision_box(
        id="table",
        size=[2.0, 2.0, 0.50],
        frame_id=BASE_LINK,
        position=[0.0, 0.0, TABLE_Z - 0.25],
        quat_xyzw=[0.0, 0.0, 0.0, 1.0],
    )
```

Listing 3: Adding the table as a no-go floor in the MoveIt 2 planning scene.

The first three `remove_collision_object` calls clean up any collision object that might still be in the scene from a previous run, which avoids stacking duplicates on top of each other. The `add_collision_box` call then publishes a $2\,\mathrm{m} \times 2\,\mathrm{m}$ slab, 50 cm thick, centred 25 cm below the table top so that the top face is exactly at `TABLE_Z`. From MoveIt 2's point of view the floor of the world has now become a solid wall, and any trajectory that would dip below the table is rejected at planning time.

6.2.2 Attaching the camera mount

The RealSense D435 is rigidly bolted to the wrist of the robot through a 3D-printed bracket. We measured the bracket with a calliper and obtained a bounding-box size of approximately $65\,\mathrm{mm} \times 115\,\mathrm{mm} \times 5\,\mathrm{mm}$. Because the bracket moves with the robot, declaring it as a static collision object would not work it has to follow every motion of the end-effector. MoveIt 2 supports exactly this case through the concept of an *attached collision object*: a collision object that is rigidly linked to one of the robot links. The function `attach_camera_mount` publishes such an object on the `/attached_collision_object` topic and parents it to the `tool0` link of the UR3e:

```
CAMERA_MOUNT_SIZE   = [0.065, 0.115, 0.005]
CAMERA_MOUNT_OFFSET = [0.0,   0.0,   -0.0025]

box = SolidPrimitive()
box.type = SolidPrimitive.BOX
box.dimensions = [float(d) for d in CAMERA_MOUNT_SIZE]

co = CollisionObject()
co.header.frame_id = END_EFFECTOR    # "tool0"
co.id = "camera_mount"
co.primitives.append(box)
co.primitive_poses.append(box_pose)
co.operation = CollisionObject.ADD

aco = AttachedCollisionObject()
aco.link_name   = END_EFFECTOR
aco.object      = co
aco.touch_links = ["tool0", "wrist_1_link",
                   "wrist_2_link", "wrist_3_link"]
pub.publish(aco)
```

Listing 4: Declaring the camera support as an attached collision object on the wrist.

The `touch_links` field is important: it tells MoveIt 2 that collisions between the bracket and the wrist links are expected and should be ignored. Without this list, the bracket would be considered as colliding with `tool0` from the moment it is attached, and *every* plan would fail. After this call, the camera support becomes a virtual extension of the wrist for collision-checking purposes: the planner now refuses any trajectory in which the bracket would scrape the body of the robot or one of the obstacles.

6.2.3 Constraining the workspace with a virtual box

As mentioned above, RRT-Connect can produce strange paths because it samples the configuration space without any preference for short or straight motions. In our setup, the most spectacular failure mode was the robot swinging the elbow far to the back of the workspace before coming forward to the grasp point. To prevent this we restricted the volume in which the robot is allowed to operate by adding a virtual *operating box* around the area we actually use. In practice this is implemented as additional collision walls (the table slab above, plus an implicit ceiling and side limits) that fence the arm into a usable cuboid. The combination of the table no-go floor, the attached camera mount, and the bounding planes is sufficient to eliminate the worst RRT-Connect detours during our test runs. We also kept the planning velocity and acceleration low (`VELOCITY_SCALE = 0.03`, i.e. 3 % of the rated values), so that any remaining unexpected motion can still be stopped by hand without damage.

6.2.4 Pre-defined poses versus the dynamic pick pose

The last preparatory step was to decide how to express each waypoint of the pick-and-place sequence. Two strategies are available with MoveIt 2: one can either provide a 6-D Cartesian goal (a position and a quaternion in `base_link`), in which case MoveIt 2 has to solve the inverse kinematics, or one can provide a full joint-space configuration, in which case the goal is unambiguous. We used both, depending on what made sense for each waypoint.

For the home pose, the place-approach pose, the safe intermediate pose above the drop zone, and the final drop pose, we hand-tuned the six joint values on the teach pendant once and stored them as constants in the script. The advantage of doing this in joint space is that the inverse kinematics solver cannot pick a different solution between two runs, which makes the place side completely repeatable. These configurations are:

```
HOME_JOINTS = [0.0, -1.57, 0.0, -1.57, 0.0, 0.0]

PLACE_APPROACH1_JOINTS = [0.0, -1.57, -1.57, -1.57, 1.57, 0.0]

PLACE_APPROACH2_JOINTS = [
    math.radians(  0), math.radians(-140), math.radians(-50),
    math.radians(-75), math.radians(  90), math.radians(  0),
]

PLACE_DROP_JOINTS = [
    math.radians(  0), math.radians(-155), math.radians(-54),
    math.radians(-58), math.radians(  90), math.radians(  0),
]
```

Listing 5: Pre-recorded joint-space poses for home and place.

For the pick side, on the other hand, we cannot pre-record anything, because the position of the shoe is by definition unknown until the detector publishes it. The pick goal must therefore be expressed as a 6-D pose in `base_link`, with the position (x, y, z) coming from the perception pipeline and the orientation given as a quaternion that points the gripper straight down at the table. The corresponding quaternion is hard-coded as `QUAT_DOWN` ($= (1, 0, 0, 0)$ in $xyzw$ form), which corresponds to a pure π-rotation around the x-axis of the base frame, so that the z-axis of the tool flange points downwards. The helper `make_pose` assembles this:

```
QUAT_DOWN = dict(x=1.0, y=0.0, z=0.0, w=0.0)

def make_pose(x: float, y: float, z: float) -> Pose:
    p = Pose()
    p.position.x = float(x)
    p.position.y = float(y)
    p.position.z = float(z)
    p.orientation.x = QUAT_DOWN["x"]
    p.orientation.y = QUAT_DOWN["y"]
    p.orientation.z = QUAT_DOWN["z"]
    p.orientation.w = QUAT_DOWN["w"]
```

```
    return p
```

Listing 6: Building the pick pose from the published (x, y) and a fixed downward quaternion.

This split between joint-space waypoints (for everything that is fixed) and Cartesian poses (for everything that depends on the detector) keeps each waypoint as simple as possible to plan.

6.3 Detailed walk-through of `pick_and_place_grasp.py`

With the planning scene prepared and the waypoints defined, the main script becomes a relatively linear state machine. We describe each stage below, with the corresponding extracts from the source code.

6.3.1 Initialisation of MoveIt 2

The script starts by initialising rclpy, creating a node and constructing a `MoveIt2` client from `pymoveit2`. The client is configured with the joint names of the UR3e, the base link, the end effector, the planning group (`ur_manipulator`) and the speed limits.

```
arm = MoveIt2(
    node=node,
    joint_names=JOINT_NAMES,
    base_link_name=BASE_LINK,          # "base_link"
    end_effector_name=END_EFFECTOR,    # "tool0"
    group_name=GROUP_NAME,             # "ur_manipulator"
)
arm.max_velocity_scaling_factor     = VELOCITY_SCALE       # 0.03
arm.max_acceleration_scaling_factor = ACCELERATION_SCALE   # 0.03
arm.planning_time                   = PLANNING_TIME_SEC    # 15.0
arm.goal_position_tolerance         = POSITION_TOL         # 1 cm
arm.goal_orientation_tolerance      = ORIENTATION_TOL      # 0.1 rad
```

Listing 7: Bootstrapping the `pymoveit2` client and configuring the planner.

A `MultiThreadedExecutor` is then started in a background thread so that the subscriptions (joint states, grasp point) remain responsive while the main thread is busy waiting for trajectories to finish.

6.3.2 Step 0 Listening to the grasp detector

Step 0 of the sequence is the actual link with the perception side. The script subscribes to `/shoe_detector/grasp_point_base`, collects `GRASP_SAMPLES = 5` valid messages, and computes the *median* of the x and y coordinates. Working with a median rather than a single sample is essential: the YOLO inference produces the occasional outlier frame in which the centroid is shifted by several centimetres, and a median over five samples filters those outliers very effectively. We also reject any message whose `frame_id` is not `base_link`, which is a contract check on the

perception side.

```python
def wait_for_grasp(node: Node):
    samples_x, samples_y, samples_z = [], [], []
    done = threading.Event()

    def _cb(msg: PointStamped) -> None:
        if msg.header.frame_id != GRASP_FRAME_EXPECTED:
            return
        samples_x.append(msg.point.x)
        samples_y.append(msg.point.y)
        samples_z.append(msg.point.z)
        if len(samples_x) >= GRASP_SAMPLES:
            done.set()

    sub = node.create_subscription(PointStamped, GRASP_TOPIC, _cb, 10)
    done.wait(timeout=GRASP_TIMEOUT_SEC)
    node.destroy_subscription(sub)

    mx = statistics.median(samples_x)
    my = statistics.median(samples_y)
    mz = statistics.median(samples_z)
    return mx, my, mz
```

Listing 8: Median filter on the grasp point published by the detector.

Once the median is computed, the script destroys the subscription and stops listening to the topic. This is a deliberate design choice: the grasp pose is *latched* for the rest of the sequence, so that a flicker in the YOLO detection during the motion cannot make the robot chase a moving target. The published z value is read but **not used** for the actual descent, because depth from the RealSense on a small dark object such as a lace hole is too noisy (± 5 to 10 mm in our experiments). Instead, we override the descent height with a fixed safe value `PICK_Z`, which we lower in 2 cm steps after verifying the TCP on the teach pendant. Before any motion is planned, the median is checked against the workspace bounds:

```python
def validate_grasp(x: float, y: float, z: float) -> bool:
    ok = True
    if not (PICK_X_MIN <= x <= PICK_X_MAX):
        print(f"   Grasp X={x:+.3f} outside bounds")
        ok = False
    if not (PICK_Y_MIN <= y <= PICK_Y_MAX):
        print(f"   Grasp Y={y:+.3f} outside bounds")
        ok = False
    return ok
```

Listing 9: Workspace sanity check on the grasp point.

If the detector returns a point outside roughly $[-0.45, 0.45]$ in either x or y, the script aborts cleanly without ever sending a plan request. This avoids long, useless planning attempts when the detector confuses a far-away background object with the target shoe.

6.3.3 Steps 1–4 The pick

The pick itself is split into four motions: home, approach, descend, and retreat. The home motion uses joint-space planning to bring the robot back to a known configuration; the approach motion uses joint-space planning towards a Cartesian goal located 20 cm above the pick; the descent uses Cartesian (straight-line) planning so that the gripper goes straight down without lateral motion; and the retreat is a Cartesian straight line back up to the approach height. The same pattern is reused for the place side. To keep the script readable we wrapped the three primitive motion calls (`move_to_pose`, `move_to_pose` with `cartesian=True`, and `move_to_configuration`) in helper functions that retry up to three times on failure, cancel any running goal first and reseed the planner from the current joint state. The pick-side helper for a Cartesian descent looks like this:

```
def move_cartesian(node, arm, pose, label, retries=3):
    for attempt in range(1, retries + 1):
        cancel_and_wait(arm)
        seed_from_current(node, arm)
        try:
            try:
                arm.move_to_pose(pose, cartesian=True)
            except TypeError:
                # older pymoveit2 without the cartesian= kwarg
                arm.move_to_pose(pose)
            if arm.wait_until_executed() is False:
                time.sleep(2.0)
                continue
            return True
        except Exception:
            time.sleep(2.0)
    return False
```

Listing 10: Cartesian motion helper with retries.

The `cancel_and_wait` call before each attempt is important: if a previous goal is still active in `move_group`, the next `move_to_pose` call will be silently rejected. The `seed_from_current` call reads the latest `/joint_states` message and forces `pymoveit2` to use those values as the start state of the next plan, instead of relying on its internal copy, which can lag behind the real robot during long sequences. Together, these two calls eliminated the most common "ghost failure" we observed during early tests, in which two consecutive plans would race against each other.

The orchestration of the pick is then a few lines of code:

```
move_home(node, arm, "home")
if not move_safe(node, arm, pose_approach, "approach"):
    abort(node, arm, spin_thread, "approach failed"); return

if not move_cartesian(node, arm, pose_pick, "pick-descend"):
    move_cartesian(node, arm, pose_approach, "pick-retreat-fail")
    abort(node, arm, spin_thread, "pick descend failed"); return

```

```
gripper_close()                        # placeholder

if not move_cartesian(node, arm, pose_approach, "pick-retreat"):
    abort(node, arm, spin_thread, "pick retreat failed"); return
```

Listing 11: Pick sequence (steps 1–4).

Note the recovery logic on the descend step: if the straight-line descent fails, the script first tries to bring the arm back up to the approach height before aborting. This avoids leaving the robot half-way down on top of the shoe.

6.3.4 Steps 5–9 The place and the return home

The place side reuses the same helpers but works entirely in joint space, using the three pre-recorded configurations introduced in Section 6.2. We move first to a safe intermediate pose (`PLACE_APPROACH1_JOINTS`), which keeps the elbow well clear of the boxes on the table, then to a pose directly above the drop location (`PLACE_APPROACH2_JOINTS`), and finally down to the drop pose itself (`PLACE_DROP_JOINTS`). After releasing the object the arm retreats back to the second approach pose and then to the home configuration, ready for the next cycle.

```
move_joints(node, arm, PLACE_APPROACH1_JOINTS, "place-approach-1")
move_joints(node, arm, PLACE_APPROACH2_JOINTS, "place-approach-2")

if not move_joints(node, arm, PLACE_DROP_JOINTS, "place-drop"):
    move_joints(node, arm, PLACE_APPROACH2_JOINTS, "place-retreat-fail")
    abort(node, arm, spin_thread, "place drop failed"); return

gripper_open()                         # placeholder

move_joints(node, arm, PLACE_APPROACH2_JOINTS, "place-retreat")
move_home(node, arm, "return-home")
```

Listing 12: Place sequence (steps 5–9).

Each `move_joints` call internally cancels any pending goal, reseeds the planner, and retries up to three times. The same abort logic as on the pick side ensures that if anything goes wrong while the object is held, the script attempts to bring the arm to a safe intermediate pose before stopping.

6.3.5 Summary of the full state machine

The complete sequence executed by the script can therefore be summarised as follows:

1. Wait for the median grasp point on `/shoe_detector/grasp_point_base`.
2. Move to the home configuration (joint space).
3. Move 20 cm above the grasp point (Cartesian goal, joint planning).

4. Descend straight down to `PICK_Z` (Cartesian straight line) and close the gripper.
5. Retreat straight up to the approach height (Cartesian straight line).
6. Move to the safe intermediate place pose (joint space).
7. Move above the drop zone (joint space).
8. Descend to the drop pose (joint space) and open the gripper.
9. Retreat to the pose above the drop zone (joint space).
10. Return to the home configuration (joint space).

6.4 Gripper control: attempted but not finished

To drive the OnRobot 2FG7 gripper that is bolted to the wrist of the UR3e, we were given two pieces of material by our professors:

- a Python module called `gripper.py`, written to open and close the 2FG7 from a ROS 2 node;
- a small URCap program to add to the program already running on the UR teach pendant, which polls a digital output of the robot controller and translates its state into a grip or release command on the gripper.

We followed the instructions and added the URCap block to the program on the teach pendant exactly as recommended. Inside that program, a background thread continuously reads the digital output `digital_out[1]`: when the bit is `True` the program calls `2FG Grip (0)` (close on the object), and when the bit is `False` it calls `2FG Release (30)` (open to a 30 mm aperture). The intention was then to control that single bit from our `pick_and_place_grasp.py` script either directly from `gripper.py`, or via a small ROS 2 service so that calling `gripper_close()` in our pipeline would simply set the bit to `True` and trigger the URCap to close the fingers.

Unfortunately, this never worked end to end. We tried several times to plug `gripper.py` and the digital-output toggling logic into our existing `pick_and_place_grasp.py`, using different approaches: a direct import of the module, wrapping it in a small ROS 2 service, and calling the digital I/O service of `ur_robot_driver` directly. None of these variants produced an actual motion of the gripper. Sometimes the import path could not be resolved because `gripper.py` was written for a slightly different workspace layout sometimes the module imported correctly but the call to the gripper would either hang or time out mid-sequence and sometimes our script reported that the digital output bit had been flipped on the controller side, yet the URCap thread on the pendant did not react and the fingers stayed where they were. At no point during our test runs did the gripper physically close or open from inside our pipeline.

Given the time we had left and the fact that an unstable gripper call could leave the robot in an unsafe state if it hung in the middle of a trajectory (object held, arm stopped halfway), we made

a pragmatic decision: rather than ship a half-broken integration that could mess up the rest of the pipeline, we chose to keep `gripper_close()` and `gripper_open()` as simple placeholders functions that print a message and sleep half a second and to focus the remaining effort on making the perception-to-motion pipeline robust end to end.

The result is that, in the current state of the project, the arm performs the full pick-and-place sequence above the shoe but does not physically grasp the object. The URCap block is in place on the teach pendant, the `gripper.py` module is in the workspace, but the bridge between our motion script and the digital output that drives the URCap is the block that is still missing for a complete demonstration. At least, with this choice, what we do have is functional and reliable: vision, frame transform, median filtering, workspace validation, planning, execution and recovery all run end to end.

6.5 Conclusion

This part of the project bridged the gap between the perception output of the YOLO-based grasp detector and the physical motion of the UR3e. We built a MoveIt 2 client on top of `pymoveit2` that subscribes to the grasp point topic, filters it with a median over several samples, validates it against a workspace envelope, and then drives the arm through a deterministic ten-step pick-and-place sequence.

A non-trivial fraction of the work went into making the planner behave well in our specific workshop. We modelled the workshop table as a thick box in the planning scene, attached the camera support to the wrist as an attached collision object so that it is taken into account in every collision check, and constrained the operating volume in order to tame the more adventurous detours of the RRT-Connect planner. The pick pose, which depends on the detector, is built from the published (x, y) together with a fixed downward quaternion and a safe descent height; the place poses, which are fixed, were pre-recorded on the teach pendant in joint space for maximum repeatability.

The most important limitation of the current implementation is the absence of a working gripper command, which we have documented honestly in Section 6.4. With that one block replaced, the system as it stands is able to perform a full vision-driven pick-and-place cycle, and we believe it provides a solid base on which to add the missing gripper control as well as further extensions, such as Cartesian place trajectories or multi-object pick-and-place.

7 Plane Removal with RANSAC

After detecting the shoe with YOLO and extracting the corresponding 3D points from the depth image, an additional filtering step was introduced to improve the quality of the reconstructed shoe point cloud. The objective of this step was to remove the supporting planar surface (table or bin bottom) that was still present in the 3D data and could negatively affect the localization of the shoe.

7.1 Objective

Although the YOLO bounding box allowed us to isolate the region of interest in the RGB image, the corresponding 3D point cloud still contained unwanted points belonging to the surrounding environment, especially the planar support surface under the shoe. These points introduced noise in the estimation of the shoe position and could reduce the accuracy of grasp planning.

To fix this issue, we introduced a plane segmentation step based on the RANSAC algorithm in order to separate the shoe from the planar surface.

7.2 RANSAC Principle

RANSAC (Random Sample Consensus) is an iterative algorithm commonly used to estimate the parameters of a mathematical model in the presence of outliers. In our case, the model to estimate was a plane corresponding to the support surface.

The algorithm operates as follows:

- Randomly sample a small subset of points from the point cloud.
- Estimate a candidate plane model from these sampled points.
- Compute how many points in the full cloud fit this plane within a given distance threshold.
- Repeat the process for multiple iterations.
- Select the plane model with the highest number of inliers.

Once the dominant plane is identified, its inlier points are removed, leaving only the non-planar points corresponding to the shoe.

The integration of RANSAC is expected to provide several advantages:

- Improved separation between the shoe and the support surface
- Reduced noise in the reconstructed point cloud
- More reliable 3D position estimation
- Better robustness for future grasp planning and robotic manipulation

7.3 Integration in the Pipeline

The RANSAC-based filtering stage was inserted after 3D point extraction and before final position estimation. The updated processing pipeline became:

1. Shoe detection in RGB image using YOLO
2. Extraction of 3D points from the depth map
3. Point cloud generation
4. Plane detection using RANSAC
5. Removal of planar inliers
6. Outlier filtering
7. Final 3D localization of the shoe

This additional step was expected to improve the robustness of the 3D localization by reducing the influence of the table or bin surface.

7.4 Implementation Approach

```python
import open3d as o3d
import numpy as np

pcd = o3d.geometry.PointCloud()
pcd.points = o3d.utility.Vector3dVector(points)

pcd = pcd.voxel_down_sample(voxel_size=0.005)

plane_model, inliers = pcd.segment_plane(
    distance_threshold=0.01,
    ransac_n=3,
    num_iterations=1000
)

[a, b, c, d] = plane_model

plane_cloud = pcd.select_by_index(inliers)
object_cloud = pcd.select_by_index(inliers, invert=True)

o3d.visualization.draw_geometries([object_cloud])
```

The implementation was carried out using the Open3D library. First, the extracted 3D points were converted into a point cloud representation. Then, a voxel downsampling step was applied to reduce noise and computational cost. RANSAC plane segmentation was subsequently used to identify the dominant planar surface. After removing the detected plane, a statistical outlier

removal filter was applied to refine the remaining point cloud. Finally, the 3D position of the shoe was estimated from the centroid of the filtered point cloud.

7.5 Limitations

Although the RANSAC-based plane removal stage was successfully designed and partially implemented, this part of the pipeline could not be fully completed and validated within the project timeframe.

The main objective was to integrate this filtering stage into the complete perception pipeline and evaluate its impact on localization accuracy. However, due to time constraints and integration difficulties with the full real-time pipeline, the experimental validation of this step was not finalized.

As a result, the RANSAC module remains a promising improvement for future work, but its quantitative performance could not be fully assessed during this project.

8 Appendix A: Commands used for the eye-in-hand calibration

8.1 Terminal 1 — UR3e Driver

```
source /opt/ros/humble/setup.bash
source ~/ur3e_ws/install/setup.bash
ros2 launch ur_robot_driver ur_control.launch.py ur_type:=ur3e robot_ip:=192.168.1.102
```

8.2 Terminal 2 — RealSense Camera

```
source /opt/ros/humble/setup.bash
source ~/ur3e_ws/install/setup.bash
ros2 launch realsense2_camera rs_launch.py enable_color:=true enable_depth:=false
    rgb_camera.color_profile:=640x480x30
```

8.3 Terminal 3 — ArUco Detection

```
source /opt/ros/humble/setup.bash
source ~/ur3e_ws/install/setup.bash
ros2 run aruco_ros single --ros-args -p marker_size:=0.100 -p marker_id:=582 -p
    reference_frame:=camera_color_optical_frame -p camera_frame:=camera_color_optical_frame
    -p marker_frame:=aruco_marker_frame -r /image:=/camera/camera/color/image_raw -r
    /camera_info:=/camera/camera/color/camera_info
```

8.4 Terminal 4 — easy_handeye2 Server

```
source /opt/ros/humble/setup.bash
source ~/ur3e_ws/install/setup.bash
ros2 launch easy_handeye2 calibrate.launch.py calibration_type:=eye_in_hand name:=ur3e_d435
    robot_base_frame:=base_link robot_effector_frame:=tool0
    tracking_base_frame:=camera_color_optical_frame tracking_marker_frame:=aruco_marker_frame
```

8.5 Terminal 5 — rqt_calibrator GUI

```
source /opt/ros/humble/setup.bash
source ~/ur3e_ws/install/setup.bash
ros2 run easy_handeye2 rqt_calibrator.py --force-discover --ros-args -p name:=ur3e_d435 -p
    calibration_type:=eye_in_hand -p robot_base_frame:=base_link -p
    robot_effector_frame:=tool0 -p tracking_base_frame:=camera_color_optical_frame -p
    tracking_marker_frame:=aruco_marker_frame
```

8.6 Terminal 6 — Detection Visualization

```
source /opt/ros/humble/setup.bash
source ~/ur3e_ws/install/setup.bash
ros2 run rqt_image_view rqt_image_view /aruco_single/result
```

Appendix B – Commands used for detection and pick-and-place

The commands below bring up the full detection and pick-and-place pipeline from a cold start. Each command runs in its own terminal, which must first source `/opt/ros/humble/setup.bash` and `~/ur3e_ws/install/setup.bash` (omitted below for brevity).

Terminal 1 – UR3e driver

Connects to the physical robot, loads our custom kinematic calibration, and publishes the TF chain rooted at `base_link`.

```
ros2 launch ur_robot_driver ur_control.launch.py \
    ur_type:=ur3e \
        robot_ip:=192.168.1.102 \
            kinematics_params_file:=$HOME/ur3e_config/my_ur3e_calibration.yaml \
                launch_rviz:=false

```

Terminal 2 – MoveIt 2 move_group

Starts the MoveIt 2 `move_group` server with the UR3e configuration and opens RViz for visualisation.

```
                ros2 launch ur_moveit_config ur_moveit.launch.py \
                    ur_type:=ur3e \
                        launch_rviz:=true

```

Terminal 3 – RealSense camera

Starts the RealSense D435 with depth aligned to colour, which is required for the back-projection performed by the grasp detector.

```
                        ros2 launch realsense2_camera rs_launch.py \
                            align_depth.enable:=true

```

Terminal 4 – Grasp detector

Runs the YOLO-based perception node and publishes the grasp point on `/shoe_detector/grasp_point_base`.

```
ros2 run shoe_detector grasp_detector --ros-args \
    -p model_path:=$HOME/ur3e_ws/src/shoe_detector/models/best_kaagle_10mars.pt \
    -p target_class:=hole \
    -p conf_threshold:=0.5
```

Terminal 5 – Static camera-to-tool transform

Republishes the result of the eye-in-hand calibration (Appendix 8) as a static transform between `tool0` and `camera_color_optical_frame`, so that the grasp detector can express its output directly in the robot `base_link` frame.

```
ros2 run tf2_ros static_transform_publisher \
    --x 0.07035858523181109 \
    --y -0.03786070717680278 \
    --z 0.03091576198336038 \
    --qx -0.006237947432436568 \
    --qy -0.0006472618464794522 \
    --qz 0.6941468239226309 \
    --qw 0.7198061238292268 \
    --frame-id tool0 \
    --child-frame-id camera_color_optical_frame
```

Terminal 6 – Verify the camera-to-base transform

Used to confirm that TF can resolve the chain from the robot base to the camera optical frame, which is a prerequisite for the grasp detector to publish in the `base_link` frame.

```
ros2 run tf2_ros tf2_echo base_link camera_color_optical_frame
```

Terminal 7 – Launch the pick-and-place script

Once everything above is up and the grasp detector is publishing, this is the command that actually starts the MoveIt 2 pick-and-place sequence described in Section 6.

```
python3
```

```
    ~/ur3e_ws/src/ur_sim/scripts/pick_and_place_grasp.py
```